Smaller Is Not Always Faster: The Memory–Performance Trade-off of LLM Quantization on an RTX 5090

Sijia Zhong **Zheng Zheng** **Haofei Sun**
Department of Computer Science and Engineering
The University of Texas at Arlington

Abstract

Quantization stores the weights of a large language model (LLM) in 8 or 4 bits instead of 16, and formats are usually compared by their bit-width. We show that the bit-width alone predicts neither the memory a format saves, nor its speed, nor its cost in quality. On one NVIDIA RTX 5090, we measure eight configurations on two inference stacks (Hugging Face Transformers with bitsandbytes, and vLLM) for five models from 3B to 32B parameters, with 1,890 timed runs, real chat traffic and three quality benchmarks. (1) 4-bit weights make a 7B model 2.60×–2.70× smaller, not 4×, and the total saving falls to 1.49× as the KV cache grows. (2) The implementation matters more than the bit-width: 8-bit weights decode 1.64× faster than BF16 as GPTQ-INT8 on vLLM, but 3.4× slower as LLM.int8() in bitsandbytes. (3) Weight-only formats lose their speedup once batching makes decoding compute-bound, as a roofline model predicts, while FP8 stays faster at every batch size. (4) 8-bit formats keep quality, and 4-bit formats cost 0.6–2.8 MMLU points, more on smaller models. Still, a larger 4-bit model beats a smaller 16-bit one: Qwen3-32B in 4 bits scores 78.1% on MMLU against 69.6% for Qwen3-8B in BF16, with only 1.20× the weight memory. Kernel benchmarks, a roofline model and GPU profiles explain each result.

1 Introduction

Large language models (LLMs) need a lot of GPU memory. The 16-bit weights of a 7B-parameter model take 15 GB, and the key–value (KV) cache of every running request comes on top. To generate each token, the GPU reads all the weights from memory, so generation is often limited by memory bandwidth rather than by arithmetic [9, 22]. Quantization stores the weights, and sometimes the activations, in 8 or 4 bits. Formats are named by their bit-width (W4A16 means 4-bit weights and 16-bit activations), which suggests a simple rule: fewer bits mean less memory, faster generation and a small loss in quality.

This rule can fail in three places. *Memory:* the embedding, the output layer, the KV cache and the activations stay in 16 bits. *Speed:* most formats must convert the weights back to 16 bits (dequantize them) before each matrix multiply; this is free only if a fast kernel hides it while the GPU waits for memory, and large batches make the GPU wait for arithmetic instead. *Quality:* the rounding error depends on the method and on the model.

Method papers report accuracy and kernel speed separately, usually on data-center GPUs [7, 8, 17, 18]; comparison studies focus on accuracy [13, 16] or on one serving stack [14]. But a user with one consumer GPU also picks a library, and the same format runs on very different kernels in Hugging Face Transformers (bitsandbytes) and in vLLM (Marlin, CUTLASS).

CSE 6359 Advanced Topics in Systems & Architecture, Fall 2026. Final project report.

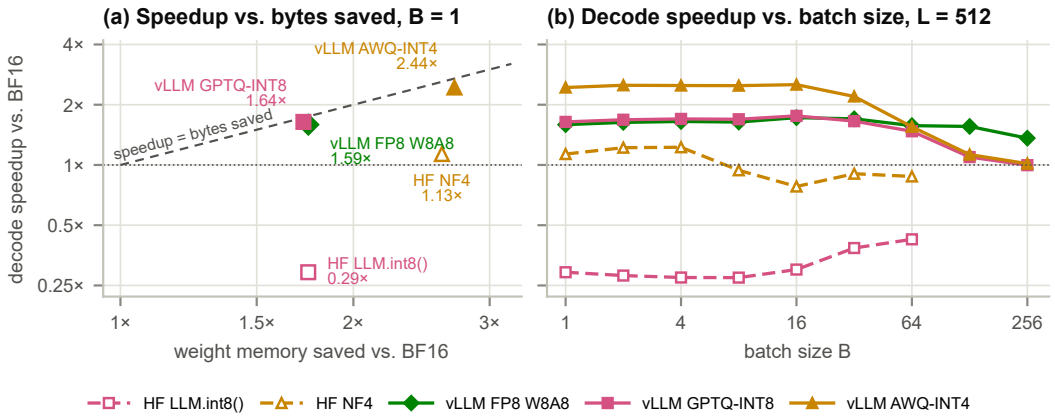


Figure 1: **The bit-width does not predict the speedup.** (a) Decode speedup over BF16 at batch size 1 against the weight memory each format saves (Qwen2.5-7B, 512-token prompts). If decode time were set only by the bytes read, every point would lie on the dashed line; the vLLM stack comes close, the HF + bitsandbytes stack does not. (b) As the batch grows, weight-only formats lose their gain; FP8 keeps it.

We measure the whole trade-off on an NVIDIA RTX 5090, a 32 GB consumer GPU, and answer the four questions of our proposal. **Q1:** How much memory does quantization really save? **Q2:** How do the time to first token (TTFT), the time per output token (TPOT) and the throughput change? **Q3:** At which batch sizes and prompt lengths does a low-bit format become faster, or slower, than BF16? **Q4:** How much quality does it cost? We test eight configurations on two backends and five models (Qwen2.5 at 3B, 7B and 14B; Qwen3-8B and Qwen3-32B to check generality), and explain the results with kernel benchmarks, a measured roofline and GPU profiles. Our main findings:

- **Memory (Q1, Section 4.1).** 4-bit weights shrink Qwen2.5-7B by 2.60×–2.70×, not 4×, and the total saving drops to 1.49× with many tokens in flight. On vLLM the saved memory becomes KV cache (1.90× more tokens).
- **Speed (Q2, Section 4.2).** The implementation decides the speedup (Figure 1a): at batch size 1, vLLM's kernels turn 90–95% of the byte saving into speed, NF4 only 44%, and LLM.int8() is 3.4× slower than BF16.
- **Crossover (Q3, Sections 4.3 and 4.4).** Weight-only formats lose their speedup where decoding becomes compute-bound (Figure 1b; AWQ-INT4 at $B \approx 175$), while FP8 wins at every batch size. On HF, kernel launches on the CPU limit small models.
- **Quality (Q4, Sections 4.6 and 4.7).** 8-bit formats stay within 0.9 points of BF16; 4-bit formats cost 0.6–2.8 MMLU points, more on smaller models, yet at a fixed memory budget a larger 4-bit model wins.

We release the harness, the raw data and the scripts that produce every number, table and figure in this report.[1]

2 Background and related work

Prefill and decode. An LLM serves a request in two phases. *Prefill* processes all L prompt tokens at once, so for a batch of B requests each linear layer multiplies its weights by $M = B \cdot L$ rows; prefill sets the TTFT. *Decode* then generates one token per request per step, so each layer sees only $M = B$ rows; one step takes one TPOT. The keys and values of earlier tokens are kept in the KV cache.

A roofline view. The roofline model [25] tells which resource limits a kernel. For a linear layer $y = xW^\top$ with $x \in \mathbb{R}^{M \times K}$, $W \in \mathbb{R}^{N \times K}$ and b bytes per weight, the arithmetic intensity (operations

[1] https://github.com/zz9tf/6359-SYS-ARCHITECT-course-work

per byte moved) is

$$I(M) \;=\; \frac{2MNK}{b\,NK + 2MK + 2MN} \;\approx\; \frac{2M}{b} \qquad (M \ll N, K). \tag{1}$$

The kernel is memory-bound when $I(M)$ is below the ridge point R (peak FLOP/s divided by memory bandwidth) and compute-bound above it. On our GPU, $R \approx 135$ FLOP/byte for BF16. Equation (1) gives three predictions, which we test in Section 4:

- **P1** Small-batch decode is memory-bound, so fewer weight bytes should give a proportional speedup, if the kernel hides the dequantization.
- **P2** Weight-only formats (W8A16, W4A16) still compute in BF16: their speedup shrinks once they reach the compute roof ($M \approx bR/2$) and is gone once BF16 is compute-bound too ($M \gtrsim R$). W8A8 formats use 8-bit tensor cores, at least twice as fast, so they stay faster.
- **P3** Prefill is compute-bound, so only W8A8 formats can shorten the TTFT.

The roofline leaves out the CPU time needed to launch each kernel. If this time exceeds the GPU time, the step is *host-bound*, and no saving of bytes helps.

Formats. LLM.int8() [5] (INT8 weights and activations, with large activation columns kept in FP16) and NF4 [6] (4-bit NormalFloat weights) come from bitsandbytes and run in Hugging Face Transformers [12]. GPTQ [7] and AWQ [17] choose the rounding with second-order or activation information and store INT4 or INT8 weights; vLLM [15] runs them with Marlin kernels [8]. FP8 [21] stores weights and activations in an 8-bit float that recent GPUs support in hardware: the floating-point form of W8A8, which SmoothQuant [26] enables for INT8.

Related work. QServe [18] notes that W4A16 kernels lose their advantage in large-batch serving; other work quantizes activations to 4 bits [2] or compresses the KV cache [11, 19]. Dettmers and Zettlemoyer [4] show that 4-bit models give the best accuracy for a fixed number of model bits. Jin et al. [13] and Li et al. [16] compare methods on accuracy, and Jin et al. note that quantization can slow inference down. Closest to us, Kurtic et al. [14] find on data-center GPUs that W4A16 suits latency-bound use and W8A8 suits high-throughput serving. We confirm this on a consumer GPU and add the Hugging Face path, memory at run time, and a kernel-level explanation of every crossover.

3 Experimental setup

Platform. We use one NVIDIA GeForce RTX 5090 (Blackwell, 170 SMs, 32 GB GDDR7, 575 W) in a workstation with an Intel Core Ultra 9 285K and 64 GB of RAM, running Ubuntu 24.04, PyTorch 2.13 (CUDA 13.0), Transformers 5.19, bitsandbytes 0.50.2 and vLLM 0.32 (CUDA graphs on). Our microbenchmarks measure 1.61 TB/s of memory bandwidth and 218 (BF16) and 645 (FP8) TFLOP/s for large matrix multiplies, which puts the BF16 ridge point at about 135 FLOP/byte.

Models and configurations. The main study uses instruction-tuned Qwen2.5 [23] at 3B, 7B and 14B parameters; Qwen3-8B and Qwen3-32B [24] test generality. Table 1 lists the eight configurations. The Hugging Face (HF) backend is what a user gets with one extra argument to `from_pretrained`: bitsandbytes quantizes the BF16 checkpoint while loading, and PyTorch runs the model eagerly. The vLLM backend serves the official Qwen GPTQ and AWQ checkpoints with Marlin kernels and converts BF16 weights to FP8 while loading. The embedding, the output layer (LM head) and the norms stay in 16 bits in every configuration.

Workloads. *Controlled sweep:* each configuration generates exactly 128 tokens for $B \in \{1, 2, 4, \ldots, 64\}$ prompts (up to 256 on vLLM) of $L \in \{128, 512, 1024, 2048\}$ random tokens, so every configuration does the same work; a smaller sweep varies the output length. *Real traffic:* 300 ShareGPT [1] conversations (median prompt 31 tokens, median answer 284 tokens), served by vLLM to 1–64 concurrent clients and by HF in static batches of 16. *Quality:* WikiText-2 [20] perplexity on 2048-token windows; GSM8K [3] on all 1,319 test questions with zero-shot chain of thought and greedy decoding; MMLU [10] on all 14,042 test questions, zero-shot. Prompts use each model's chat template, and Qwen3 runs in non-thinking mode.

Metrics and statistics. TTFT is the time to the first output token, TPOT the mean time of each later token, and throughput the number of output tokens per second. Each cell has one warm-up and

Table 1: The eight configurations. W/A: bits of the weights and of the activations in the matrix multiply.

Backend	Name	W/A	Method	Kernel	Weights from
HF	BF16	16/16	—	cuBLAS	BF16 checkpoint
HF	INT8	8/8†	LLM.int8()	bitsandbytes, cuBLASLt	quantized on load
HF	NF4	4/16	NF4, block 64	bitsandbytes	quantized on load
vLLM	BF16	16/16	—	cuBLAS	BF16 checkpoint
vLLM	FP8	8/8	FP8 E4M3	CUTLASS	quantized on load
vLLM	GPTQ-INT8	8/16	GPTQ, group 128	Marlin	Qwen checkpoint
vLLM	AWQ-INT4	4/16	AWQ, group 128	Marlin	Qwen checkpoint
vLLM	GPTQ-INT4	4/16	GPTQ, group 128	Marlin	Qwen checkpoint

† Activation columns with values above 6.0 are multiplied in FP16.

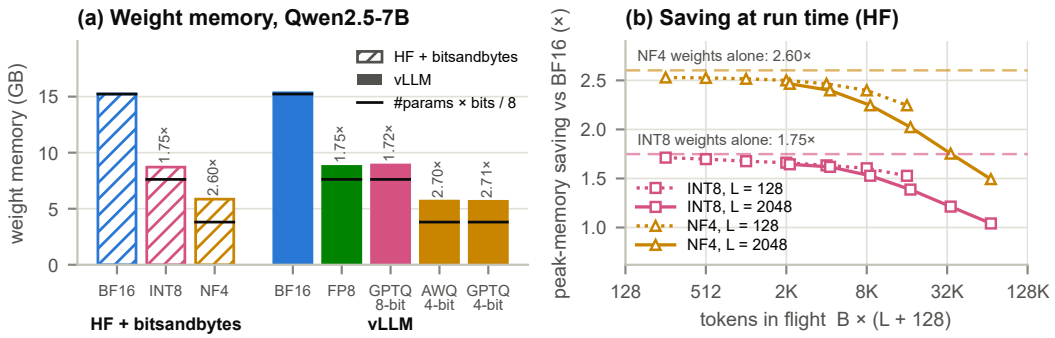


Figure 2: **Q1: the real saving is smaller than the bit-width suggests.** (a) Weight memory of Qwen2.5-7B measured after loading (bars) against the naive estimate #params×bits/8 (black ticks); labels give the saving against BF16 on the same backend. (b) Saving of the peak memory on HF as more tokens are in flight (dashed: the saving of the weights alone).

three measured runs with fresh prompts; the run-to-run variation of the end-to-end time is below 0.7% in 95% of the cells and 2.3% at most. Memory and energy come from PyTorch and NVML. For quality, all configurations answer the same questions, so we compare each with HF BF16 by an exact McNemar test, with a Bonferroni correction over the 18 comparisons of each benchmark ($\alpha = 0.0028$).

4 Results

4.1 Q1: How much memory does quantization actually save?

Answer: *Less than the bit-width suggests, and less as the context grows. On vLLM, the saved memory becomes room for more requests.*

Weights. The naive estimate is too low for every format, by up to 57% for NF4 (Figure 2a). Only the decoder's linear layers are quantized: the embedding and the LM head (14% of Qwen2.5-7B's parameters) stay in 16 bits, and each format stores scales. Counting these parts predicts every HF measurement within 0.6% (Appendix A). So 4-bit weights shrink the 7B model by 2.60× (NF4) to 2.70× (AWQ-INT4), and 8-bit weights by 1.75×.

Run time. The KV cache and the activations are not quantized, so the saving shrinks as more tokens are in flight (Figure 2b). NF4 cuts the peak memory of one short request by 2.53×, but that of 32 requests of 2048 tokens by only 1.49×; LLM.int8(), which keeps extra copies of the activations, by 1.04×. On HF, the 7B model runs out of memory at $B = 64$, $L = 2048$ in every format, because the prefill activations fill the GPU.

Capacity. vLLM turns all the memory the weights leave free (within 85% of the GPU) into KV cache. With AWQ-INT4, Qwen2.5-7B holds 1.90× as many cached tokens as with BF16 (355K vs. 187K). The 14B model is the extreme case. In BF16 its 29.5 GB of weights leave no room for a KV

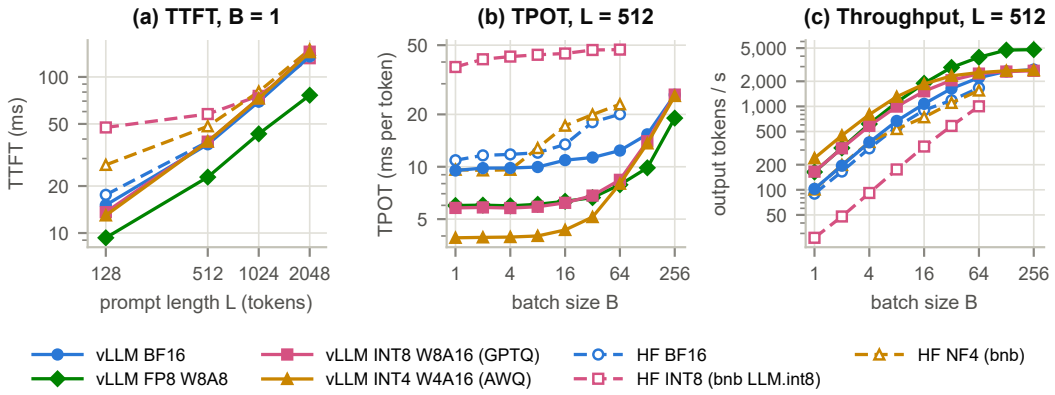


Figure 3: **Q2: latency and throughput of Qwen2.5-7B** with 128 generated tokens. (a) TTFT of one request; (b) TPOT and (c) output throughput for 512-token prompts. Solid lines: vLLM; dashed lines with hollow markers: HF with bitsandbytes.

cache, even when vLLM may use 95% of the GPU, so vLLM cannot serve it at all. In AWQ-INT4 it runs with 82K cached tokens.

4.2 Q2: How do latency and throughput change?

Answer: *With fused kernels, fewer bytes give an almost proportional decode speedup for small batches; with bitsandbytes they do not. No weight-only format shortens prefill.*

Decode. At batch size 1 (prediction P1), vLLM's BF16 step takes 9.52 ms, within 99% of the time needed to read the 15.2 GB of weights, so it is purely memory-bound. On vLLM, fewer bytes do mean less time (Figure 3b): AWQ-INT4 decodes 2.44× faster than BF16, GPTQ-INT8 1.64× and FP8 1.59×, which is 90–95% of their byte savings. On HF the same idea fails: LLM.int8() is 3.4× slower than BF16, and NF4 is 1.13× faster on 7B (1.22× on 14B) but 1.39× slower on 3B. These are whole stacks: LLM.int8() and GPTQ-INT8 differ in method, activation path and engine, not only in kernel. Section 4.4 separates the kernels from the rest.

Prefill. As P3 predicts, AWQ and GPTQ change the TTFT by at most 7%, and NF4 makes it 1.26× slower, because bitsandbytes dequantizes every weight matrix first (Figure 3a). FP8, which also quantizes the activations, makes a 2048-token prefill 1.77× faster.

Throughput. At $B = 256$ (Figure 3c), AWQ-INT4 (2,720 tokens/s) no longer beats BF16 (2,766), while FP8 reaches 4,794 tokens/s, 1.73× the BF16 rate. Energy per token follows latency (Appendix B).

4.3 Q3: When does lower precision pay off?

Answer: *Weight-only formats pay off for small batches, and their gain ends near the batch size at which decoding becomes compute-bound. FP8 pays off everywhere; the bitsandbytes formats rarely do.*

Figure 4 maps the end-to-end speedup over batch size and prompt length. Three regimes appear. (i) *Weight-only formats with fused kernels* (AWQ, GPTQ) win by up to 2.44× when decode dominates. Their gain shrinks as the batch and the prompts grow, and AWQ-INT4 stops winning at $B \approx 175$ for 512-token prompts, close to $M \approx R \approx 135$, where BF16 decoding itself becomes compute-bound (P2). (ii) *FP8 W8A8* wins in every cell (1.59×–1.80×), because it halves the bytes *and* raises the compute roof. (iii) *bitsandbytes formats* rarely win: LLM.int8() is slower in every cell (0.27×–0.71×), and NF4 wins only for 7B below $B \approx 7$. Longer answers raise the share of decode and push the crossover up: at $B = 16$, AWQ-INT4 speeds up a request with 32 output tokens by 1.26×, but one with 512 output tokens by 2.16× (Appendix B).

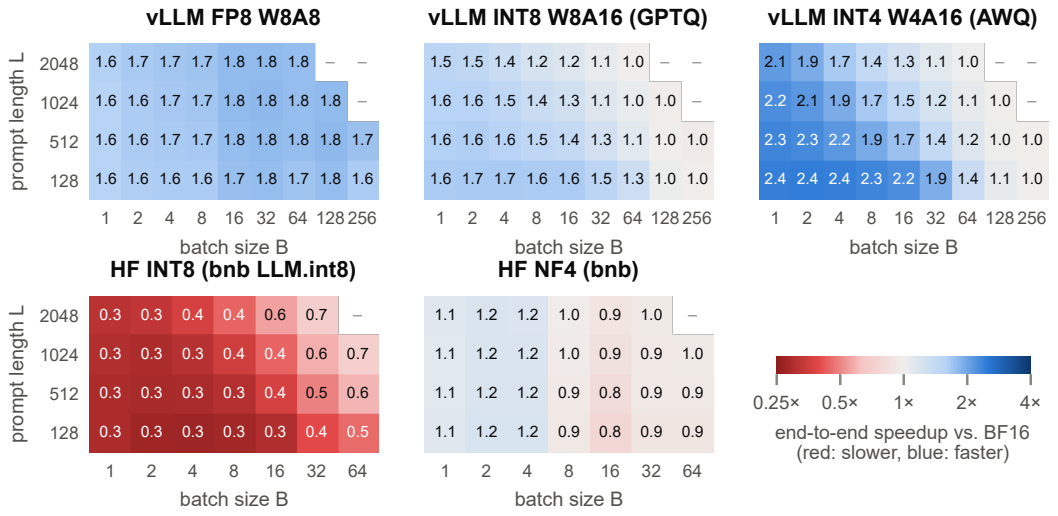


Figure 4: **Q3: where each format pays off.** End-to-end speedup of B requests (L-token prompts, 128 generated tokens) over BF16 on the same backend, Qwen2.5-7B. Dashes: out of memory or not run. GPTQ-INT4 behaves like AWQ-INT4 (Table 6 in Appendix B).

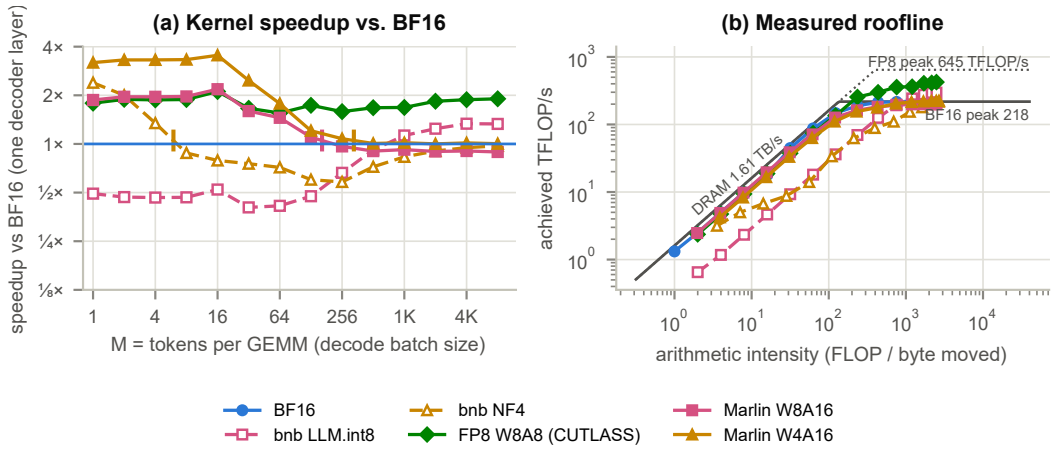


Figure 5: **Why the crossover happens.** (a) Speedup of each low-bit kernel over BF16 cuBLAS on the matrix shapes of one Qwen2.5-7B decoder layer, against the tokens M per multiply (the decode batch size); ticks mark where it falls below 1.05. (b) The same kernels on the measured roofline.

4.4 Why: kernels, the roofline and host overhead

Answer: *The roofline explains vLLM, where the kernels are fast. The CPU cost of launching kernels explains HF, where the GPU is often idle.*

Kernels. We time each low-bit kernel alone on the matrix shapes of one 7B decoder layer (Figure 5a). Marlin W4A16 is 3.2× faster than BF16 at $M = 1$ and keeps a gain above 5% up to $M \approx 333$; Marlin W8A16 starts at 1.9× and ends at $M \approx 164$. Figure 5b shows why (P2): fewer bytes per weight move a kernel up the bandwidth roof faster, but it hits the same BF16 compute roof. Real kernels approach the roof gradually, so the crossovers lie somewhat above the ridge point. FP8 W8A8 is 1.8× faster at $M = 1$ and still 1.9× at $M = 8192$, because its compute roof is about three times higher (645 vs. 218 TFLOP/s). The bitsandbytes kernels are not slow everywhere: NF4's kernel is 2.39× faster than BF16 at $M = 1$ (it dequantizes whole matrices for larger M), yet the whole NF4 model is only 1.13× faster. Time must be lost outside the kernels.

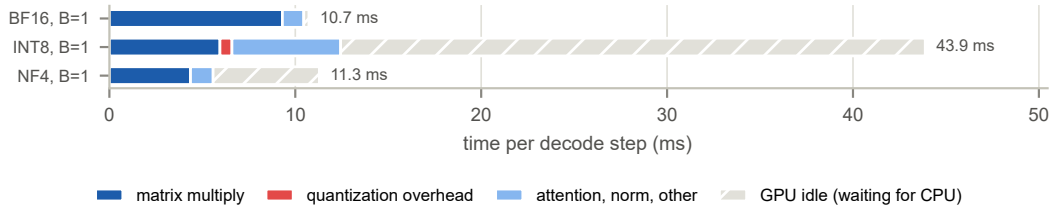


Figure 6: **Where an HF decode step goes** (Qwen2.5-7B, 512-token prompts): GPU time by kernel type, and the time the GPU waits for the CPU (hatched).

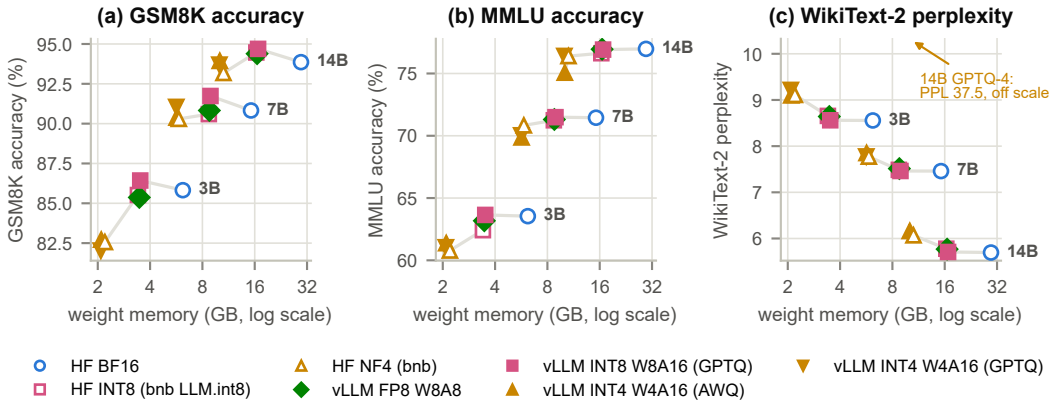


Figure 7: **Q4: quality against weight memory** for every Qwen2.5 configuration. Lines connect the configurations of one model size; hollow markers: HF, filled: vLLM. A larger model in 4 bits beats a smaller model in 16 bits at less memory.

Host overhead. Figure 6 profiles one HF decode step of the 7B model. With BF16, the GPU is busy 97% of the step. LLM.int8() launches 3.7× as many kernels (casts, outlier handling, quantization, two matrix multiplies per layer), and the GPU is busy only 28% of the time: Python and kernel launches set the speed. NF4 lies in between (49% busy). More layers mean more launches, so the 36-layer 3B model suffers more than the 28-layer 7B model. vLLM avoids most of this cost with CUDA graphs, which replay a decode step with very few launches.

4.5 Do the results hold for real traffic?

Answer: *Yes, and the serving system matters even more than the number format.* In a replay of 300 ShareGPT requests (Table 7 in Appendix C), AWQ-INT4 gives the lowest TPOT with one client (4.1 ms vs. 9.6 ms for BF16), and FP8 the highest throughput with 64 clients (5,388 vs. 3,537 tokens/s), as the crossover of Section 4.3 predicts. With the same 16 requests in flight, vLLM BF16 delivers 5.3× the throughput of HF BF16. Both run BF16, so this gap belongs to the serving stack (continuous [27] vs. static batching, CUDA graphs), not to quantization; the two stacks are also timed differently (Appendix C).

4.6 Q4: How much quality does quantization cost?

Answer: *8-bit formats keep quality. 4-bit formats cost a small but real amount, more on smaller models. At a fixed memory budget, a larger 4-bit model is still the better choice.*

Noise floor. Table 2 reports perplexity, multi-step math (GSM8K) and knowledge (MMLU). Even BF16 on two backends disagrees: the kernels round differently, and greedy decoding turns tiny differences into different reasoning chains. vLLM and HF disagree on 19 of the 7B model's GSM8K answers, yet their accuracies differ by only 0.4 points (MMLU: less than 0.1). Differences this small are not caused by quantization.

Table 2: Quality cost of each format. First row: BF16 values (WikiText-2 perplexity; GSM8K and MMLU accuracy in %). Other rows: change against HF BF16 of the same model (accuracy in points). **Bold**: significant after Bonferroni correction over the 18 comparisons per benchmark (exact McNemar test). Last row: BF16 on vLLM, the noise floor.

			Qwen2.5-3B			Qwen2.5-7B			Qwen2.5-14B		
Format	Backend	W/A	PPL	GSM8K	MMLU	PPL	GSM8K	MMLU	PPL	GSM8K	MMLU
BF16 (value)	HF	16/16	8.56	85.8	63.5	7.46	90.8	71.4	5.70	93.9	77.0
LLM.int8()	HF	8/8	+0.10	−0.3	**−1.1**	+0.03	−0.2	−0.2	+0.08	+0.6	−0.3
FP8	vLLM	8/8	+0.09	−0.5	−0.4	+0.05	0.0	−0.1	+0.07	+0.5	0.0
GPTQ-INT8	vLLM	8/16	0.00	+0.6	+0.1	0.00	+0.9	0.0	+0.01	+0.8	−0.1
NF4	HF	4/16	+0.55	**−3.3**	**−2.8**	+0.31	−0.5	**−0.6**	+0.37	−0.7	−0.6
AWQ-INT4	vLLM	4/16	+0.54	**−3.1**	**−2.2**	+0.39	−0.5	**−1.6**	+0.46	+0.1	**−1.9**
GPTQ-INT4	vLLM	4/16	+0.66	**−3.8**	**−2.5**	+0.32	+0.2	**−1.4**	+31.77‡	−0.2	−0.6
BF16 (noise)	vLLM	16/16	0.00	+0.2	+0.1	0.00	+0.4	0.0	–	–	–

‡Large only on raw text; on chat-formatted text this checkpoint matches the other 14B formats (Section 4.6).

8-bit formats keep quality. GPTQ-INT8 and FP8 stay within 0.9 points of BF16 on both benchmarks for all three sizes. On GSM8K, the only 8-bit differences with $p < 0.05$ are improvements (GPTQ-INT8 on 7B, $+0.9$ points, $p = 0.043$; GPTQ-INT8 on 14B, $+0.8$ points, $p = 0.013$), and neither survives the Bonferroni correction. The only measurable 8-bit cost is LLM.int8() on the 3B model (-1.1 MMLU points, $p < 0.001$).

4-bit formats cost quality, most on small models. The 4-bit formats change MMLU accuracy by -2.8 to -2.2 points on 3B, -1.6 to -0.6 on 7B and -1.9 to -0.6 on 14B; 3 of 3, 3 of 3 and 1 of 3 of these drops stay significant after the correction. On GSM8K, only the 3B model clearly loses accuracy (-3.8 to -3.1 points). A smaller model has less spare capacity to absorb the rounding error.

Model size matters more than bit-width. Qwen2.5-14B in AWQ-INT4 needs 10.1 GB of weights, less than the 15.2 GB of Qwen2.5-7B in BF16, and scores 93.9% vs. 90.8% on GSM8K and 75.1% vs. 71.4% on MMLU (McNemar $p < 0.001$ for both). At a fixed memory budget, parameters are worth more than bits (Figure 7), as the scaling laws of Dettmers and Zettlemoyer [4] predict.

One metric is not enough. The 14B GPTQ-INT4 checkpoint matches the other 14B formats on GSM8K (93.7%) and MMLU (-0.6 points), but its WikiText-2 perplexity is 37.5 instead of 5.71–6.16. The effect is real (it remains with BF16 activations and appears at every position). The checkpoint ranks the correct next token first for only 37% of raw WikiText tokens (other 14B formats: 60–62%), but for 77% of chat-formatted GSM8K answers (others: 77%): it keeps the chat format it serves but has lost much of its general language modeling. Task benchmarks alone would miss this; perplexity alone would reject a checkpoint that works for chat.

4.7 Do the results generalize? Qwen3-8B and Qwen3-32B

Answer: *The systems results carry over to a newer model family. The quality order of the formats does not, so it must be measured for each model.*

Qwen3-8B has 36 layers and 8 KV heads (Qwen2.5-7B: 28 and 4) and runs in non-thinking mode. The systems results carry over (Figure 8a, Appendix E): at batch size 1, AWQ-INT4 decodes 2.33× faster than BF16 (Qwen2.5-7B: 2.44×), FP8 1.54×, and LLM.int8() runs at 0.28× of the BF16 speed. The one difference supports the host-overhead explanation: NF4, 1.13× faster on the 28-layer Qwen2.5-7B, runs at 0.90× of the BF16 speed on the 36-layer Qwen3-8B. Quality costs stay small (Figure 8b; no change above 0.7 points). But the order of the formats changes: FP8 costs 0.1 MMLU points on Qwen2.5-7B but 0.7 on Qwen3-8B, as much as AWQ-INT4. Which format keeps quality depends on the model.

Qwen3-32B raises a different question: does the model fit at all? Its BF16 weights need 66 GB and 8-bit weights about half of that, more than the GPU's 32 GB, so it runs only in 4 bits (AWQ-INT4: 19.6 GB plus 29K cached tokens; NF4: 20.7 GB). Compared with Qwen3-8B in BF16 (16.4 GB;

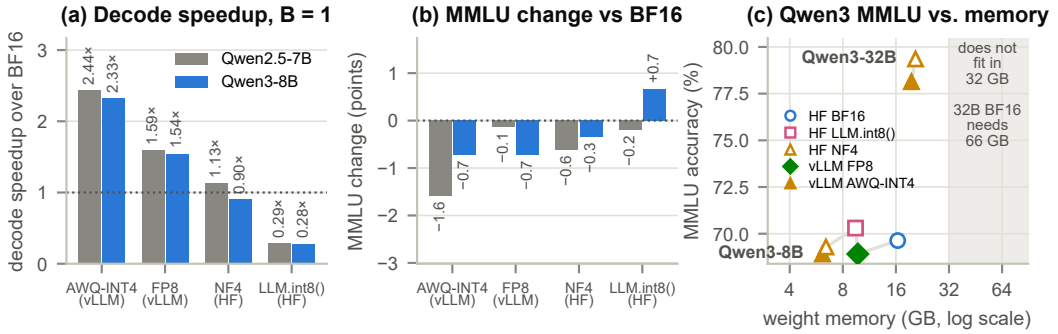


Figure 8: **Generalization.** (a) Decode speedup and (b) MMLU change against BF16 for Qwen2.5-7B and Qwen3-8B. (c) Qwen3 MMLU accuracy against weight memory: Qwen3-32B fits only in 4 bits, and it beats Qwen3-8B in BF16.

Table 3: A decision guide for quantized LLM inference on one GPU.

Situation	Choice	Evidence
Planning memory	Use the real ratio, not $16/b$	4-bit saves only 2.60×–2.70× (§4.1)
Model does not fit	4-bit weights	Qwen3-32B fits 32 GB only in 4 bits (§4.7)
Few users ($B \lesssim 16$)	AWQ/GPTQ-INT4, fused kernels	2.44× faster decode (§4.2)
Many users ($B \gtrsim 64$)	FP8 W8A8	Faster at every B; 1.73× throughput (§4.3)
Fixed memory budget	Larger model in 4 bits	14B AWQ-INT4 beats 7B BF16 (§4.6)
Small model, accuracy first	8-bit (FP8, GPTQ-INT8)	4-bit costs up to 2.8 MMLU points (§4.6)
Only HF + bitsandbytes	Quantize for memory, not speed	LLM.int8() is 3.4× slower (§4.4)
New checkpoint	Check perplexity *and* tasks	One checkpoint fails only on raw text (§4.6)

Figure 8c), it needs 13.3 vs. 10.3 ms per token at batch size 1 and scores 94.2% vs. 91.4% on GSM8K ($p < 0.001$) and 78.1% vs. 69.6% on MMLU ($p < 0.001$). A model four times larger, using only 1.20× the weight memory, is significantly better on both benchmarks. On this GPU, that model can run only because of quantization. (With the checkpoint's FP16 activations, 13 of 146 perplexity windows overflow; BF16 activations fix this.)

5 Discussion

A decision guide. Table 3 turns the results into rules; in each, the kernel and the regime decide, not the bit-width.

Lessons for system design. (1) Weight-only formats save bandwidth but not compute; in the compute-bound regime only formats that tensor cores run natively (FP8 here, FP4 on the newest GPUs) help. (2) The software stack matters as much as the format. (3) With 4-bit weights, the KV cache dominates memory at long contexts, so KV-cache quantization [11, 19] is the natural next step.

Limitations. We test one GPU and dense Qwen models. The mechanism is architectural and Qwen3 reproduces it, but crossover batch sizes depend on each GPU's ratio of compute to bandwidth. The GPU also drives a display, so we could not lock its clocks (repeated runs still agree closely). We do not study 4-bit activations, KV-cache quantization or mixture-of-experts models.

6 Conclusion

On one consumer GPU, the bit-width alone predicts neither the memory a quantization format saves, nor its speed, nor its quality cost. Memory savings are real but smaller than advertised. Speed depends on the implementation and the regime: fused weight-only kernels win at small batches, FP8 at every batch size. Quality costs grow as bits and model size shrink, yet a larger 4-bit model beats a smaller 16-bit one. Smaller is not always faster, but with the right kernel in the right regime, it is both.

Author contributions

Sijia Zhong built the harness and the memory study (Q1), Zheng Zheng ran the sweeps and the kernel and profiling analysis (Q2, Q3), and Haofei Sun ran the real-traffic and quality evaluations (Q4). All three analyzed the results and wrote the report and the slides.

References

[1] ShareGPT_Vicuna_unfiltered (ShareGPT v3, cleaned split). Hugging Face Datasets, https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered, 2023.

[2] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[3] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[4] T. Dettmers and L. Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In *International Conference on Machine Learning (ICML)*, 2023.

[5] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2208.07339.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.14314.

[7] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.17323.

[8] E. Frantar, R. L. Castro, J. Chen, T. Hoefler, and D. Alistarh. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. In *Proceedings of the 30th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 239–251, 2025. doi: 10.1145/3710848.3710871.

[9] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer. AI and memory wall. *IEEE Micro*, 44(3):33–39, 2024.

[10] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

[11] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] Hugging Face. Bitsandbytes quantization. Transformers documentation, https://huggingface.co/docs/transformers/quantization/bitsandbytes, 2026.

[13] R. Jin, J. Du, W. Huang, W. Liu, J. Luan, B. Wang, and D. Xiong. A comprehensive evaluation of quantization strategies for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[14] E. Kurtic, A. Marques, S. Pandit, M. Kurtz, and D. Alistarh. "Give Me BF16 or Give Me Death"? Accuracy-Performance trade-offs in LLM quantization. *arXiv preprint arXiv:2411.02355*, 2024.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[16] S. Li, X. Ning, L. Wang, T. Liu, X. Shi, S. Yan, G. Dai, H. Yang, and Y. Wang. Evaluating quantized large language models. In *International Conference on Machine Learning (ICML)*, 2024.

[17] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. arXiv:2306.00978.

[18] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. arXiv:2405.04532.

[19] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024.

[20] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1609.07843.

[21] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[22] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.

[23] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[24] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[25] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[26] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.10438.

[27] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

A Additional memory results

Figure 9 and Table 4 give the weight memory of every configuration and model size. Figure 10 splits the peak memory of the 7B model on HF into weights, KV cache and activations. It shows why the run-time saving shrinks: as the batch grows, the KV cache and the activations take a growing share of the peak memory, and neither is quantized. Figure 11 shows the KV-cache capacity that vLLM gets from the memory each format frees.

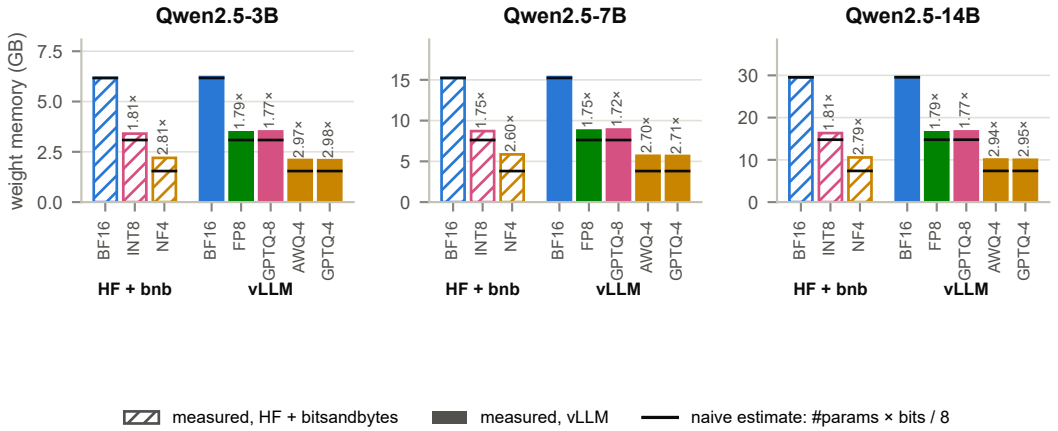


Figure 9: Weight memory of every configuration for the three Qwen2.5 sizes (bars: measured; black ticks: #params $\times$ bits/8). Labels give the saving against BF16 on the same backend.

Table 4: Weight memory: the naive estimate (#params$\times$bits/8), our model (embedding, LM head and norms stay in 16 bits; each format adds its scales), and the value measured on the GPU.

Model	Config	Naive (GB)	Model (GB)	Measured (GB)	Ratio vs BF16	Naive ratio
Qwen2.5-7B	HF BF16	15.23	15.23	15.23	1.00×	1.00×
Qwen2.5-7B	HF INT8 (LLM.int8)	7.62	8.71	8.71	1.75×	2.00×
Qwen2.5-7B	HF NF4	3.81	5.85	5.85	2.60×	4.00×
Qwen2.5-7B	vLLM BF16	15.23	15.23	15.34	1.00×	1.00×
Qwen2.5-7B	vLLM FP8 W8A8	7.62	8.71	8.77	1.75×	2.00×
Qwen2.5-7B	vLLM GPTQ W8A16	7.62	8.86	8.90	1.72×	2.00×
Qwen2.5-7B	vLLM AWQ W4A16	3.81	5.57	5.68	2.70×	4.00×
Qwen2.5-7B	vLLM GPTQ W4A16	3.81	5.57	5.66	2.71×	4.00×
Qwen2.5-14B	HF BF16	29.54	29.54	29.54	1.00×	1.00×
Qwen2.5-14B	HF INT8 (LLM.int8)	14.77	16.33	16.34	1.81×	2.00×
Qwen2.5-14B	HF NF4	7.39	10.55	10.58	2.79×	4.00×
Qwen2.5-14B	vLLM BF16	29.54	29.54	29.60	1.00×	1.00×
Qwen2.5-14B	vLLM FP8 W8A8	14.77	16.33	16.51	1.79×	2.00×
Qwen2.5-14B	vLLM GPTQ W8A16	14.77	16.64	16.73	1.77×	2.00×
Qwen2.5-14B	vLLM AWQ W4A16	7.39	9.98	10.07	2.94×	4.00×
Qwen2.5-14B	vLLM GPTQ W4A16	7.39	9.98	10.02	2.95×	4.00×

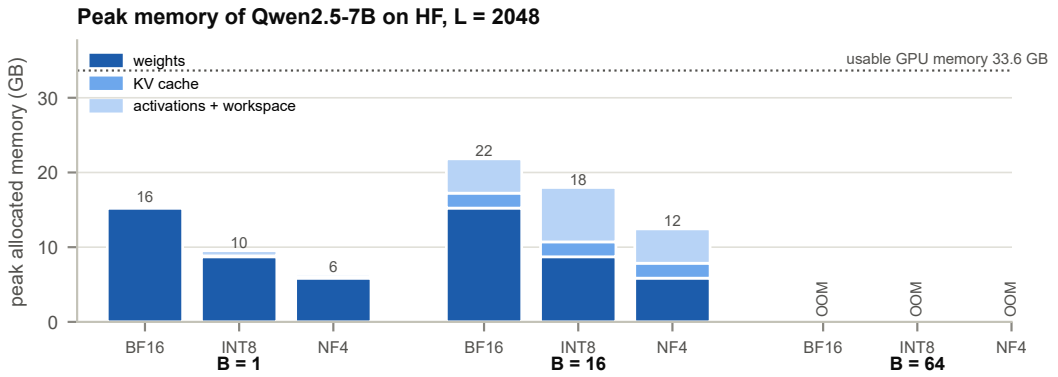


Figure 10: Peak allocated memory of Qwen2.5-7B on HF with 2048-token prompts, split into weights, KV cache and activations. OOM: out of memory.

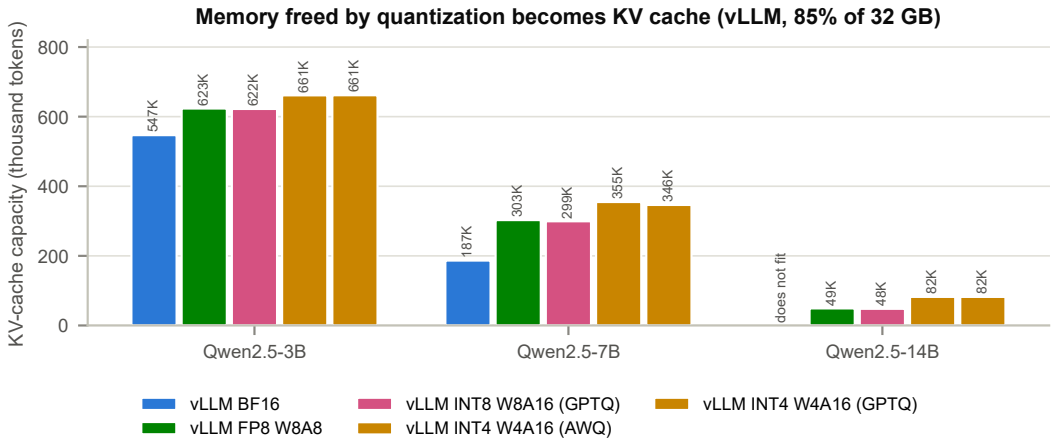


Figure 11: KV-cache capacity that vLLM allocates with 85% of the GPU after loading the weights.

B Additional speed results

Table 5 lists the latency, throughput and energy numbers behind Figure 3, and Table 6 lists where each format stops paying off (Figure 4). Figure 12 shows the energy per output token. GPU power stays close to the board limit in every configuration (410–579 W), so energy per token follows latency. LLM.int8() draws little power (164 W) but runs so long that it uses the most energy per token. Figure 13 shows how the answer length moves the speedup (Section 4.3).

Table 5: Latency, throughput and energy of Qwen2.5-7B with 512-token prompts and 128 output tokens (mean of 3 runs). Subscripts give the batch size. TTFT and TPOT in ms; Speedup_1 is the decode speedup over BF16 on the same backend; energy is GPU energy per output token.

Config	TTFT_1	TPOT_1	TPOT_{64}	tok/s_{64}	Speedup_1	mJ/tok_1	mJ/tok_{64}
HF BF16	38.4	10.90	20.04	1669	1.00×	5092	294
HF INT8 (LLM.int8)	57.8	37.45	47.14	1003	0.29×	6187	305
HF NF4	48.3	9.62	22.87	1550	1.13×	3188	340
vLLM BF16	37.0	9.52	12.39	2209	1.00×	4878	249
vLLM FP8 W8A8	22.8	5.99	7.88	3886	1.59×	2507	141
vLLM GPTQ W8A16	38.5	5.80	8.41	2471	1.64×	2588	234
vLLM AWQ W4A16	38.6	3.90	7.99	2544	2.44×	1919	224
vLLM GPTQ W4A16	36.9	3.88	7.89	2642	2.45×	1840	219

Table 6: Where each format stops paying off (Qwen2.5-7B, 512-token prompts, 128 output tokens). Speedup over BF16 on the same backend at batch size 1 and at the largest batch we ran (in brackets), and the batch size up to which the format is faster (linear interpolation in $\log B$).

	Decode step (TPOT)			Whole request (E2E)		
Config	B=1	largest B	faster up to	B=1	largest B	faster up to
HF INT8 (LLM.int8)	0.29×	0.43× (64)	never	0.30×	0.60× (64)	never
HF NF4	1.13×	0.88× (64)	$B \approx 7$	1.12×	0.93× (64)	$B \approx 7$
vLLM FP8 W8A8	1.59×	1.36× (256)	all B	1.59×	1.73× (256)	all B
vLLM GPTQ W8A16	1.64×	1.00× (256)	$B \approx 251$	1.61×	0.97× (256)	$B \approx 123$
vLLM AWQ W4A16	2.44×	1.02× (256)	all B	2.33×	0.98× (256)	$B \approx 175$
vLLM GPTQ W4A16	2.45×	1.05× (256)	all B	2.35×	1.03× (256)	all B

Qwen2.5-3B. Figures 14 and 15 repeat the latency and crossover analysis for the 3B model. On vLLM the picture is the same as for 7B. On HF the smaller model is worse off. Its BF16 step is already limited by the host (8.3 ms per step against a memory floor of 3.8 ms), so fewer bytes cannot help: NF4 never beats BF16, and LLM.int8() falls to 0.13× of the BF16 speed, because the 36-layer model launches more kernels per step than the 28-layer 7B model.

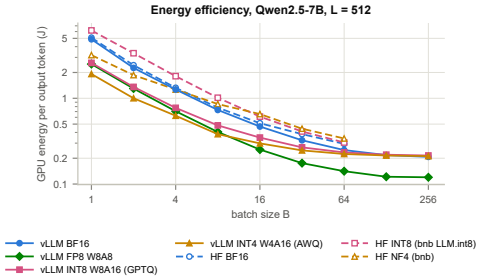


Figure 12: GPU energy per output token (NVML), Qwen2.5-7B, 512-token prompts.

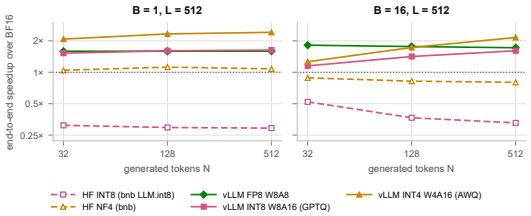


Figure 13: End-to-end speedup over BF16 against the number of generated tokens (Qwen2.5-7B, 512-token prompts). Longer answers raise the share of decode, and with it the gain of weight-only formats.

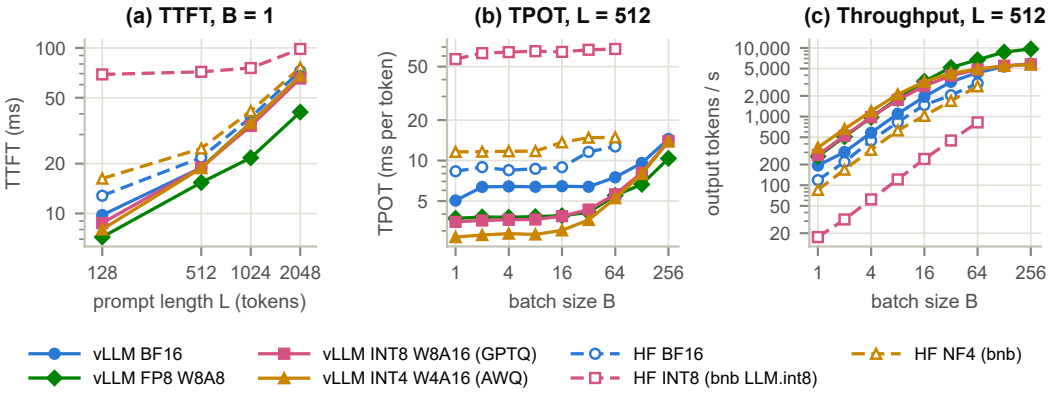


Figure 14: Latency and throughput of Qwen2.5-3B (same layout as Figure 3).

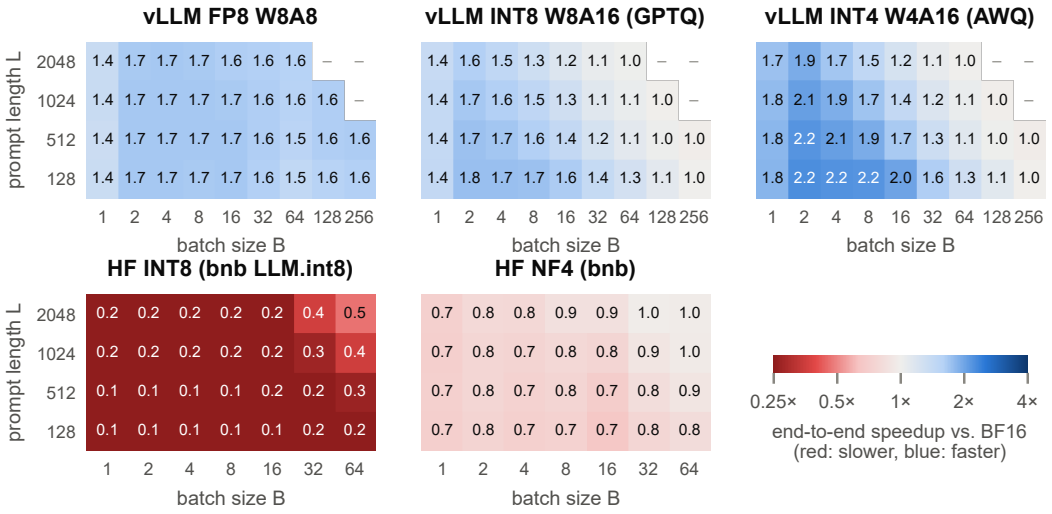


Figure 15: End-to-end speedup over BF16 for Qwen2.5-3B (same layout as Figure 4).

C Real-traffic details

Table 7 summarizes the ShareGPT replay of Section 4.5. The HF and vLLM numbers compare deployment stacks, not formats, and are timed differently. vLLM is measured by `vllm bench serve` at the client, after 8 warm-up requests; its TTFT includes the server's queueing and HTTP overhead. HF has no warm-up, and its TTFT is the time from the start of a batch's `generate` call to its first token; it leaves out the time a request waits for earlier batches, which favors HF. Even so, HF's median TTFT is 797 ms against 34 ms for vLLM. Figure 16 shows the lengths of the 300 requests, and Figure 17 and Table 8 give the full serving results, including the 99th-percentile latencies.

Table 7: Real traffic: 300 ShareGPT requests replayed against Qwen2.5-7B with their reference answer lengths. C is the number of concurrent requests (HF runs static batches of 16). Output tokens per second, and median TTFT and TPOT in ms.

	$C = 1$		$C = 16$		$C = 64$	
Config	tok/s	TPOT	tok/s	TTFT	tok/s	TPOT
HF BF16 (static)	–	–	254	797	–	–
HF INT8 (static)	–	–	119	691	–	–
HF NF4 (static)	–	–	215	842	–	–
vLLM BF16	104	9.6	1,334	34	3,537	13.6
vLLM FP8	164	6.1	2,237	21	5,388	8.9
vLLM GPTQ-8	166	6.0	2,166	22	4,733	10.5
vLLM AWQ-4	240	4.1	3,020	18	5,180	10.1
vLLM GPTQ-4	241	4.1	2,970	18	5,181	10.0

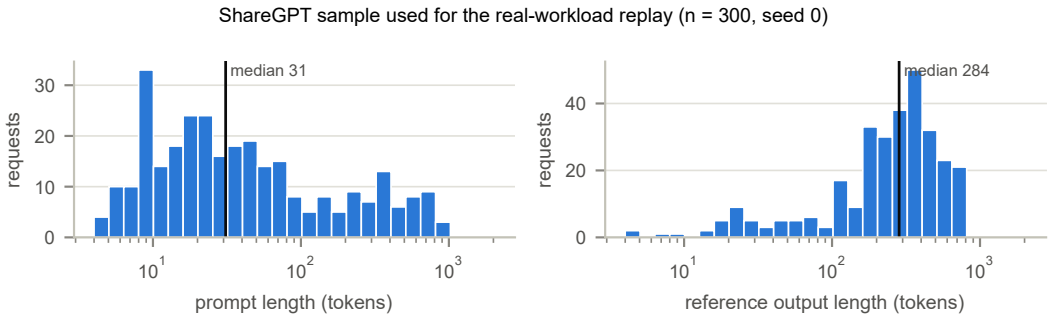


Figure 16: Prompt and answer lengths of the 300 ShareGPT requests used in the replay.

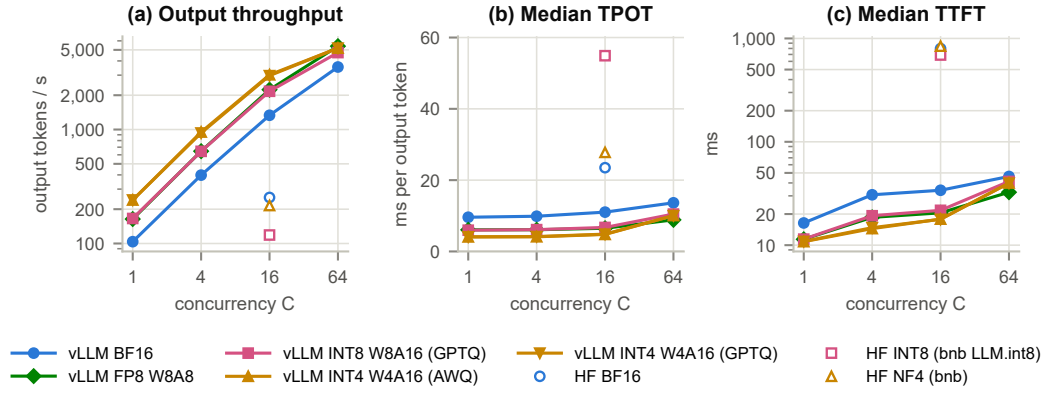


Figure 17: The ShareGPT replay against Qwen2.5-7B: vLLM at 1 to 64 concurrent clients (lines) and HF with static batches of 16 (hollow markers). TTFT and TPOT are medians over requests.

Table 8: All serving results: 300 ShareGPT requests replayed against Qwen2.5-7B (reference answer lengths). C: concurrent clients for vLLM, static batch size for HF. Throughput in requests/s and output tokens/s; TTFT and TPOT in ms (median / 99th percentile).

Config	C	req/s	tok/s	TTFT p50 / p99	TPOT p50 / p99
HF BF16 (static)	16	0.84	254	797 / 1244	23.5 / 29.7
HF INT8 (static)	16	0.39	119	691 / 1158	54.9 / 64.1
HF NF4 (static)	16	0.72	215	842 / 1271	27.8 / 35.4
vLLM BF16	1	0.35	104	16 / 62	9.6 / 9.6
vLLM BF16	64	11.76	3537	46 / 1654	13.6 / 16.7
vLLM FP8	1	0.54	164	11 / 44	6.1 / 6.2
vLLM FP8	64	17.92	5388	33 / 1485	8.9 / 15.1
vLLM GPTQ-8	1	0.55	166	11 / 63	6.0 / 6.2
vLLM GPTQ-8	64	15.74	4733	41 / 1743	10.5 / 18.0
vLLM AWQ-4	1	0.80	240	11 / 65	4.1 / 4.5
vLLM AWQ-4	64	17.23	5180	40 / 1747	10.1 / 14.8
vLLM GPTQ-4	1	0.80	241	11 / 62	4.1 / 4.4
vLLM GPTQ-4	64	17.23	5181	40 / 1722	10.0 / 19.3

D Full quality results

Table 9 lists every quality number with its p-value, for all five models.

Table 9: All quality results. WikiText-2 perplexity (2048-token windows), GSM8K accuracy (%, 1,319 questions, zero-shot chain of thought, greedy) and MMLU accuracy (%, 14,042 questions, zero-shot). Δ is the change against HF BF16 of the same model (Qwen3-32B has no BF16 run: it does not fit the GPU); p is the two-sided exact McNemar test on the paired answers.

Model	Config	PPL	ΔPPL	GSM8K	Δ	p	MMLU	Δ	p
Qwen2.5-3B	HF BF16	8.56	–	85.8	–	–	63.5	–	–
	HF INT8	8.66	+0.10	85.5	−0.3	0.777	62.4	−1.1	<0.001
	HF NF4	9.11	+0.55	82.6	−3.3	<0.001	60.8	−2.8	<0.001
	vLLM BF16	8.56	0.00	86.0	+0.2	0.871	63.7	+0.1	0.075
	vLLM FP8	8.64	+0.09	85.4	−0.5	0.610	63.2	−0.4	0.038
	vLLM GPTQ-8	8.56	0.00	86.4	+0.6	0.322	63.6	+0.1	0.261
	vLLM AWQ-4	9.10	+0.54	82.7	−3.1	0.002	61.4	−2.2	<0.001
	vLLM GPTQ-4	9.22	+0.66	82.0	−3.8	<0.001	61.1	−2.5	<0.001
Qwen2.5-7B	HF BF16	7.46	–	90.8	–	–	71.4	–	–
	HF INT8	7.49	+0.03	90.6	−0.2	0.771	71.2	−0.2	0.134
	HF NF4	7.77	+0.31	90.3	−0.5	0.500	70.8	−0.6	0.002
	vLLM BF16	7.46	0.00	91.2	+0.4	0.359	71.4	0.0	0.704
	vLLM FP8	7.51	+0.05	90.8	0.0	1.000	71.3	−0.1	0.327
	vLLM GPTQ-8	7.46	0.00	91.7	+0.9	0.043	71.5	0.0	0.610
	vLLM AWQ-4	7.85	+0.39	90.4	−0.5	0.576	69.9	−1.6	<0.001
	vLLM GPTQ-4	7.78	+0.32	91.1	+0.2	0.828	70.0	−1.4	<0.001
Qwen2.5-14B	HF BF16	5.70	–	93.9	–	–	77.0	–	–
	HF INT8	5.78	+0.08	94.5	+0.6	0.280	76.6	−0.3	0.025
	HF NF4	6.07	+0.37	93.2	−0.7	0.281	76.4	−0.6	0.003
	vLLM FP8	5.77	+0.07	94.4	+0.5	0.371	76.9	0.0	0.862
	vLLM GPTQ-8	5.71	+0.01	94.7	+0.8	0.013	76.9	−0.1	0.416
	vLLM AWQ-4	6.16	+0.46	93.9	+0.1	1.000	75.1	−1.9	<0.001
	vLLM GPTQ-4	37.46[‡]	+31.77	93.7	−0.2	0.899	76.4	−0.6	0.005
Qwen3-8B	HF BF16	9.73	–	91.4	–	–	69.6	–	–
	HF INT8	9.72	0.00	92.0	+0.6	0.382	70.3	+0.7	<0.001
	HF NF4	9.98	+0.25	91.5	+0.2	0.909	69.3	−0.3	0.156
	vLLM BF16	9.72	0.00	92.0	+0.6	0.215	69.6	0.0	0.610
	vLLM FP8	9.73	0.00	91.8	+0.5	0.488	68.9	−0.7	<0.001
	vLLM AWQ-4	10.18	+0.46	91.1	−0.2	0.818	68.9	−0.7	0.003
Qwen3-32B	HF NF4	7.93	–	92.6	–	–	79.4	–	–
	vLLM AWQ-4	7.88[§]	–	94.2	–	–	78.1	–	–

[‡]High only on raw text, not on chat-formatted text (Section 4.6). [§]Run with BF16 activations: with the checkpoint's FP16 activations, some 2048-token windows overflow (Section 4.7).

E Generalization details

Table 10 compares Qwen2.5-7B and Qwen3-8B on the same grid.

Table 10: The main results for Qwen2.5-7B and Qwen3-8B (non-thinking mode) on the same grid (512-token prompts, 128 output tokens). Throughput is compared at the largest batch at which both models' KV caches hold every request ($B = 64$). Qwen3-8B has no official GPTQ checkpoint.

Metric (vs. BF16)	Qwen2.5-7B	Qwen3-8B
Weight saving, AWQ-INT4 (vLLM)	2.70×	2.67×
Weight saving, NF4 (HF)	2.60×	2.55×
Decode speedup $B = 1$, AWQ-INT4	2.44×	2.33×
Decode speedup $B = 1$, FP8	1.59×	1.54×
Decode speedup $B = 1$, NF4	1.13×	0.90×
Decode speedup $B = 1$, LLM.int8()	0.29×	0.28×
Throughput $B = 64$, AWQ-INT4	1.15×	1.12×
Throughput $B = 64$, FP8	1.76×	1.67×
GSM8K Δ pp (p), FP8	0.0 (1.000)	+0.5 (0.488)
GSM8K Δ pp (p), LLM.int8()	−0.2 (0.771)	+0.6 (0.382)
GSM8K Δ pp (p), AWQ-INT4	−0.5 (0.576)	−0.2 (0.818)
GSM8K Δ pp (p), NF4	−0.5 (0.500)	+0.2 (0.909)
MMLU Δ pp (p), FP8	−0.1 (0.327)	−0.7 (<0.001)
MMLU Δ pp (p), LLM.int8()	−0.2 (0.134)	+0.7 (<0.001)
MMLU Δ pp (p), AWQ-INT4	−1.6 (<0.001)	−0.7 (0.003)
MMLU Δ pp (p), NF4	−0.6 (0.002)	−0.3 (0.156)

F Reproducibility

Every number in this report is computed from the raw measurements by `scripts/make_numbers.py`, and the figures and tables come from `make_figures.py` and `make_tables.py`. `run_all.sh` runs all experiments again (about twelve GPU-hours on an RTX 5090). A job queue ran the experiments one at a time, and only while no other user's process was on the GPU. Rows written while the GPU was shared (2 windows, 26 minutes in total, logged in `results/excluded_windows.jsonl`) were discarded and those jobs were run again. Re-measuring vLLM BF16 on the 7B model after the interruption reproduced the original TPOT within 0.5%.